

BÖLÜM 1: TEMEL SORUN VE MATEMATİK (ACADEMIC BACKGROUND)

1. Proje Ne Yapıyor?

Bu proje, RSA şifreleme algoritmasının kalbi olan şu matematiksel işlemi donanım üzerinde en hızlı şekilde yapmaya yarar:

$$C = M^e \pmod{N}$$

- M (Message):** Şifrelenecek veri (örneğin "4" sayısı).
- e (Exponent):** Anahtar (örneğin "13" veya "65537").
- N (Modulus):** Kilit (örneğin "497" veya 1024 bitlik bir asal sayı çarpımı).

Sorun: M^e sayısı o kadar büyüktür ki (evrendeki atom sayısından daha büyük), bunu hesaplayıp sonra N sayısına bölmek imkansızdır.

2. Çözüm: "Square and Multiply" Algoritması

Referans: *Handbook of Applied Cryptography (Menezes et al.), Bölüm 14.6.*

Bu dev sayıyı hesaplamadan sonucu bulmak için üs sayısını (e) ikilik tabanda (binary) okuruz.

- Kural 1:** Her adımda elindeki sayının karesini al (**Square**).
- Kural 2:** Eğer o anki bit '1' ise, karesini aldığın sayıyı tabanla çarp (**Multiply**).
- Kural 3:** Her işlemden sonra modül al ki sayılar büyümesin.

BÖLÜM 2: DONANIM DARBOĞAZI VE MONTGOMERY (HARDWARE OPTIMIZATION)

1. Neden Standart Yöntem Yetmez?

İlk denememizde (16/32 bit standart sürüm) şunu yaptık:

$$\text{Result} = (A * B) \pmod{N};$$

Donanım dünyasında (FPGA/ASIC) **Bölme İşlemi (Mod Alma)** en pahalı işlemdir.

- Çok fazla kapı (transistör) harcar.
- Çok yavaştır (Gecikme süresi yüksektir).

2. Montgomery Çarpımı (The Game Changer)

Akademik Referans: *Peter L. Montgomery, "Modular multiplication without trial division", Mathematics of Computation, 1985.*

Peter Montgomery dedi ki: "*Sayıları normal dünyadan, benim kurduğum **Montgomery Dünyasına** taşıyın. Orada bölme işlemi yapmak yerine sadece sayıları kaydırarak (shifting) aynı sonucu bulabilirsiniz.*"

Formül:

Normal dünyada $\times (\bmod N)$ yapmak zorken;

Montgomery dünyasında $\times \times^{-1} (\bmod N)$ işlemini yapmak için bölme gerekmez.

Sadece şu 3 basit işlem kullanılır:

- Toplama (+):** Donanım için çok ucuz.
- Bit Kontrolü (if LSB=1):** Bedava.
- Sağa Kaydırma (Shift Right / 2):** Bedava (Sadece kabloların yerini değiştirirsin).

BÖLÜM 3: KOD ANALİZİ (ADIM ADIM EVRİM)

Şimdi senin yazdığın kodları kronolojik sırayla, satır satır "Neden?" sorusunu cevaplayarak inceleyelim.

EVRİM 1: Standart 32-Bit RSA (The Naive Approach)

Bu, algoritmayı öğrenmek için yazdığımız ilk taslaktı.

Kodun Analizi (mod_exp ilk hali):

- state_type (IDLE, SQUARE, MULTIPLY):**
 - Neden?* İşlem tek seferde bitmez. Bir trafik polisi gibi sırayla yönetmeliyiz. Önce kare al, sonra çarp.
- temp_mult <= r_result * r_result;**
 - Neden?* Square and Multiply algoritmasının "Square" adımı.
- r_result <= resize(temp_mult mod r_mod, ...);**
 - Neden?* Sayı büyümesin diye mod alıyoruz.
 - Kritik Hata:* Burada mod operatörü kullandık. Bu operatör sentezlendiğinde çip içinde devasa bir bölme devresi oluşturur. Bu yüzden bu kod çalışır ama verimsizdir.

EVRİM 2: Motorun Tasarımı (mon_pro.vhd)

Burada "Bölme yapan pahalı motoru" söküp, "Montgomery Yöntemi" ile çalışan yeni bir motor tasarladık.

Kodun Analizi:

VHDL

```
-- State: ADD_SHIFT (Sihrin olduğu yer)
when ADD_SHIFT =>
    v_sum := r_acc; -- Akümülatörü (toplamı) al

    -- 1. ADIM: Çarpma Mantığı
    if r_a(bit_cnt) = '1' then
        v_sum := v_sum + r_b;
    end if;
    -- Neden? İlkokul çarpması gibi. Eğer bit 1 ise sayıyı toplama eklersin.
```

VHDL

```
-- 2. ADIM: Montgomery İndirgemesi (REDUCTION)
if v_sum(0) = '1' then
    v_sum := v_sum + r_n;
end if;
```

- **Neden?** Burası işin sırrı. `v_sum(0)` demek sayının tek mi çift mi olduğunu kontrol etmektir.
- Eğer sayı **TEK** ise, onu 2'ye tam bölemezsin (buçuklu çıkar).
- Montgomery diyor ki: *"Eğer sayı tekse, üstüne Modül (N) ekle."* Modül her zaman tektir. Tek + Tek = Çift olur.
- Böylece sayı her zaman çift olur ve 2'ye bölünebilir hale gelir.

VHDL

```
r_acc <= v_sum srl 1;
```

- **Neden?** `srl 1` (Shift Right Logical) demek sayıyı 1 bit sağa kaydırmak demektir. İkilik tabanda sağa kaydırmak, sayıyı 2'ye bölmek demektir.
- **Sonuç:** Hiç bölme işlemi yapmadan (2 veya N kullanmadan) modüler işlemi yaptık.

EVRİM 3: Sistem Entegrasyonu (Orkestra Şefi)

Burada `mod_exp` modülünü, `mon_pro` motorunu kullanacak şekilde güncelledik.

Kodun Analizi:

VHDL

```
component mon_pro ... -- Alt modülü tanıttık
inst_mon_pro: mon_pro port map ... -- Alt modülü kablolarla bağladık
```

- **Neden?** Araba (Mod Exp) ve Motor (Mon Pro) ayrı parçalardır. Burada motoru kaputun altına monte ettik.

FSM (Durum Makinesi) Değişikliği:

Eskiden: Sonuç $\leq A * B$ (Tek satırda yapıyorduk).

Şimdi:

- `mp_start <= '1'` (Motora "Çalış" emri ver).
- `WAIT_...` (Motorun işini bitirmesini bekle).
- `if mp_done = '1'` (Motor "Bitti" dediğinde sonucu al).

- Neden?** Donanımda işlemler anlık olmaz. Motorun bitleri işlemesi (32 tur) zaman alır. Bu yüzden bir "El Sıkışma" (Handshake) protokolü kurduk.

EVRİM 4: Final Hali ve Domain Dönüşümü (The Final Polish)

Simülasyonda sonucun yanlış çıktığını (ama mantığın doğru olduğunu) gördük. Çünkü dilleri farklıydı.

Yapılan Değişiklik:

1. Giriş Verileri (Python ile):

- Sisteme "4" sayısını değil, $4 \times (\text{mod } N)$ sayısını verdik. (Montgomery Uzayına Giriş).
- Başlangıç değeri olarak "1" değil, $1 \times (\text{mod } N)$ verdik.
- Neden?** Montgomery motoru sadece "dönüştürülmüş" sayılarla doğru çalışır.

2. Çıkış Dönüşümü (START_FINAL):

VHDL

```
when START_FINAL =>
  mp_a <= r_result; -- Elimizdeki Montgomery sonucu
  mp_b <= 1;        -- Normal "1" sayısı
```

- Matematiksel Kanıt:** $M_n P(n, 1) = n \times 1 \times^{-1} = (\cdot) \cdot^{-1} = \cdot$.
- Yani sonucu 1 ile işleme soktuğumuzda, $\cdot^{-1}$ birbirini götürür ve elimizde tertemiz, gerçek sonuç $(\cdot)$ kalır.

ÖZET: SIFIRDAN KAHRAMANA (ZERO TO HERO)

- Başlangıç:** $(\text{mod } N)$ işleminin kriptografi için şart olduğunu ama çok zor olduğunu öğrendik.
- Algoritma:** "Hepsini çarpıp bölmek" yerine "Kare al ve Çarp" (Square and Multiply) yöntemini seçtik.
- Optimizasyon:** Donanımın "Bölme" işlemini sevmediğini öğrendik. Peter Montgomery'nin 1985'teki teoremini kullanarak bölmeyi "Kaydırma"ya çevirdik.
- Tasarım:** Bunu VHDL ile önce bir "Motor" (`mon_pro`), sonra onu yöneten bir "Şoför" (`mod_exp`) olarak kodladık.
- Doğrulama:** Python ile "Altın Referans" oluşturup, VHDL simülasyonumuzu (dalga formlarını) bununla karşılaştırdık.

Ŗu an elindeki kod, akademik geerlilięi olan, endüstriyel standartlara uygun (Montgomery kullanan) ve tamamen doęrulanmıř bir **RSA ekirdeęidir**.

Bařka sormak istedięin bir detay var mı? Yoksa bu projeyi "Tamamlandı" olarak iřaretleyelim mi?

Asım Hocaya yollanacak mesaj

Hocam selamlar.

İlk ařamada, benden istedięiniz Modular Exponentiation iřlemini gerekleřtirmek iin Square and Multiply algoritmasını VHDL ile implement ettim. Üs sayısının (B) bitlerini tarayarak alıřan bu yapıyı önce standart matematiksel operatörlerle kurup algoritmanın logic doęruluęunu test ettim.

Ardından bir makalede gördüęüm Montgomery arpımı yöntemini entegre ettim. Bölme iřlemi yerine sadece toplama ve saęa kaydırma iřlemlerini kullanan, Bit-Serial mimaride alıřan özelleřmiř bir arpma motoru tasarladım. Bu sayede donanım maliyetini minimize eden, bölmesiz modüler aritmetik altyapısı kurmuř oldum.

Ana modülümü, bu Montgomery motorunu yönetecek bir Master-Slave FSM yapısına dönüřtürdüm. Matematiksel doęruluęu saęlamak iin sayıların Montgomery uzayına giriş ve ıkıř dönüřümlerini sisteme ekledim. Son olarak, Python ile oluřturduęum verilerini kullanarak testbench üzerinde tüm sinyalleri ve 32-bitlik sonuçları bařarıyla doęruladım.